```
# scikit baata confusion matrix ra classification report
```

```
from tensorflow.keras.layers import Conv2D
```

```
test = "/content/drive/MyDrive/AI&ML/FruitinAmazon/FruitinAmazon/test"
train = "/content/drive/MyDrive/AI&ML/FruitinAmazon/FruitinAmazon/train"
```

```
filters = 32
kernel_size = (3, 3)
layer = Conv2D(filters, kernel_size, strides=(1,1), padding="valid", activation=None, use_bias=True,
kernel_initializer="glorot_uniform")
```

```
from tensorflow.keras.layers import MaxPooling2D, AveragePooling2D
max_pool = MaxPooling2D(pool_size=(2,2), strides=None, padding="valid")
avg_pool = AveragePooling2D(pool_size=(2,2), strides=None, padding="valid")
```

```
import tensorflow as tf
# Define image size and batch size
img_height = 128 # Example image height
img_width = 128 # Example image width
batch_size = 32
validation_split = 0.2 # 80% training, 20% validation
# Create a preprocessing layer for normalization
rescale = tf.keras.layers.Rescaling(1./255) # Normalize pixel values to [0, 1]
# Create training dataset with normalization
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
train,
labels='inferred',
label_mode='int',
image_size=(img_height, img_width),
interpolation='nearest',
batch_size=batch_size,
shuffle=True,
validation_split=validation_split,
subset='training',
seed=123
)
# Apply the normalization (Rescaling) to the dataset
train_ds = train_ds.map(lambda x, y: (rescale(x), y))
# Create validation dataset with normalization
val_ds = tf.keras.preprocessing.image_dataset_from_directory(
train,
labels='inferred',
label_mode='int',
image_size=(img_height, img_width),
interpolation='nearest',
batch_size=batch_size,
shuffle=False,
validation_split=validation_split,
subset='validation',
seed=123
)
# Apply the normalization (Rescaling) to the validation dataset
val_ds = val_ds.map(lambda x, y: (rescale(x), y))
```

```
Found 90 files belonging to 6 classes.
Using 72 files for training.
Found 90 files belonging to 6 classes.
Using 18 files for validation.
```

Sample from pg 8 (ENd to End CNN model)

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np
# Load a sample dataset (MNIST for simplicity)
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
# Normalize and reshape data
x_train = x_train.astype("float32") / 255.0
```

```
x_test = x_test.astype("float32") / 255.0
x_train = np.expand_dims(x_train, axis=-1) # Add channel dimension
x_test = np.expand_dims(x_test, axis=-1)
# Define a simple CNN model
model = keras.Sequential([
layers.Conv2D(32, (3, 3), activation="relu", input_shape=(28, 28, 1)),
layers.MaxPooling2D((2, 2)),
layers.Conv2D(64, (3, 3), activation="relu"),
layers.MaxPooling2D((2, 2)),
layers.Flatten(),
layers.Dense(128, activation="relu"),
layers.Dense(10, activation="softmax") # 10 classes for MNIST digits
])
# Compile the model
model.compile(optimizer="adam",

loss="sparse_categorical_crossentropy",
metrics=["accuracy"])

# Train the model
model.fit(x_train, y_train, epochs=5, batch_size=32, validation_data=(x_test, y_test))
# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test accuracy: {test_acc:.4f}")
# Make predictions
predictions = model.predict(x_test[:5])
predicted_labels = np.argmax(predictions, axis=1)
print("Predicted labels:", predicted_labels)
print("Actual labels: ", y_test[:5])
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 66s 34ms/step - accuracy: 0.9617 - loss: 0.1268 - val_accuracy: 0.9874 - val_loss: 0.0420
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 80s 33ms/step - accuracy: 0.9866 - loss: 0.0426 - val_accuracy: 0.9868 - val_loss: 0.0384
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 83s 34ms/step - accuracy: 0.9909 - loss: 0.0280 - val_accuracy: 0.9909 - val_loss: 0.0284
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 81s 33ms/step - accuracy: 0.9934 - loss: 0.0202 - val_accuracy: 0.9886 - val_loss: 0.0372
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 83s 34ms/step - accuracy: 0.9951 - loss: 0.0151 - val_accuracy: 0.9904 - val_loss: 0.0307
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.9904 - loss: 0.0307
Test accuracy: 0.9904
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 93ms/step
Predicted labels: [7 2 1 0 4]
Actual labels:  [7 2 1 0 4]
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential([

    # Convolutional Layer 1
    Conv2D(filters=32, kernel_size=(3,3), strides=1, padding='same', input_shape=(28,28,1)),

    # Activation
    tf.keras.layers.ReLU(),

    # Pooling Layer 1
    MaxPooling2D(pool_size=(2,2), strides=2),

    # Convolutional Layer 2
    Conv2D(filters=32, kernel_size=(3,3), strides=1, padding='same'),

    # Activation
    tf.keras.layers.ReLU(),

    # Pooling Layer 2
    MaxPooling2D(pool_size=(2,2), strides=2),

    # Flatten
    Flatten(),

    # Fully Connected Layers
```

```
    Dense(64, activation='relu'),
    Dense(128, activation='relu'),

    # Output Layer
    Dense(10, activation='softmax')
])

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 28, 28, 32)	320
re_lu (ReLU)	(None, 28, 28, 32)	0
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_4 (Conv2D)	(None, 14, 14, 32)	9,248
re_lu_1 (ReLU)	(None, 14, 14, 32)	0
max_pooling2d_4 (MaxPooling2D)	(None, 7, 7, 32)	0
flatten_1 (Flatten)	(None, 1568)	0
dense_2 (Dense)	(None, 64)	100,416
dense_3 (Dense)	(None, 128)	8,320
dense_4 (Dense)	(None, 10)	1,290

Total params: 119,594 (467.16 KB)
Trainable params: 119,594 (467.16 KB)
Non-trainable params: 0 (0.00 B)

```
model.compile(optimizer="adam",

loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
```

```
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping

#Callbacks
checkpoint = ModelCheckpoint(
    'best_cnn_model.h5',
    monitor='val_loss',
    save_best_only=True,
    verbose=1
)

earlystop = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True,
    verbose=1
)

callbacks = [checkpoint, earlystop]

#Training
history = model.fit(
    x_train, y_train,
    batch_size=16,
    epochs=250,
    validation_data=(x_test, y_test),  # assuming this is your val_ds
    callbacks=callbacks
)
```

```
Epoch 7: val_loss improved from 0.03288 to 0.03200, saving model to best_cnn_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format

Epoch 7: finished saving model to best_cnn_model.h5
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 75s 20ms/step - accuracy: 0.9952 - loss: 0.0143 - val_accuracy: 0.9908 - val_loss: 0.0320
Epoch 8/250
3748/3750 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9967 - loss: 0.0101
Epoch 8: val_loss did not improve from 0.03200
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 75s 20ms/step - accuracy: 0.9959 - loss: 0.0124 - val_accuracy: 0.9899 - val_loss: 0.0352
Epoch 9/250
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9968 - loss: 0.0099
Epoch 9: val_loss did not improve from 0.03200
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 75s 20ms/step - accuracy: 0.9965 - loss: 0.0109 - val_accuracy: 0.9906 - val_loss: 0.0362
Epoch 10/250
3748/3750 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9973 - loss: 0.0092
Epoch 10: val_loss did not improve from 0.03200
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 75s 20ms/step - accuracy: 0.9967 - loss: 0.0110 - val_accuracy: 0.9899 - val_loss: 0.0380
Epoch 11/250
3747/3750 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9980 - loss: 0.0066
Epoch 11: val_loss did not improve from 0.03200
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 74s 20ms/step - accuracy: 0.9974 - loss: 0.0088 - val_accuracy: 0.9909 - val_loss: 0.0366
Epoch 12/250
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9970 - loss: 0.0084
Epoch 12: val_loss did not improve from 0.03200
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 83s 20ms/step - accuracy: 0.9972 - loss: 0.0090 - val_accuracy: 0.9913 - val_loss: 0.0366
Epoch 13/250
3748/3750 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9982 - loss: 0.0057
Epoch 13: val_loss did not improve from 0.03200
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 74s 20ms/step - accuracy: 0.9977 - loss: 0.0078 - val_accuracy: 0.9892 - val_loss: 0.0498
Epoch 14/250
3749/3750 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9977 - loss: 0.0072
Epoch 14: val_loss did not improve from 0.03200
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 75s 20ms/step - accuracy: 0.9974 - loss: 0.0084 - val_accuracy: 0.9906 - val_loss: 0.0414
Epoch 15/250
3747/3750 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9984 - loss: 0.0048
Epoch 15: val_loss did not improve from 0.03200
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 75s 20ms/step - accuracy: 0.9980 - loss: 0.0061 - val_accuracy: 0.9897 - val_loss: 0.0568
Epoch 16/250
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9976 - loss: 0.0073
Epoch 16: val_loss did not improve from 0.03200
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 76s 20ms/step - accuracy: 0.9976 - loss: 0.0078 - val_accuracy: 0.9873 - val_loss: 0.0753
Epoch 17/250
3749/3750 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9980 - loss: 0.0066
Epoch 17: val_loss did not improve from 0.03200
3750/3750 ━━━━━━━━━━━━━━━━━━━━ 75s 20ms/step - accuracy: 0.9980 - loss: 0.0070 - val_accuracy: 0.9900 - val_loss: 0.0519
Epoch 17: early stopping
```

```
# 🔹 Evaluate on test data
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)

print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")
```

```
313/313 - 3s - 10ms/step - accuracy: 0.9908 - loss: 0.0320
Test Loss: 0.0320
Test Accuracy: 0.9908
```

```
model.save('saved_model.h5')
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
```

```
loaded_model = tf.keras.models.load_model('saved_model.h5')
test_loss, test_acc = loaded_model.evaluate(x_test, y_test, verbose=2)

print(f"Loaded Model Test Accuracy: {test_acc:.4f}")
```

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty unt
313/313 - 3s - 10ms/step - accuracy: 0.9908 - loss: 0.0320
Loaded Model Test Accuracy: 0.9908
```

```
predictions = loaded_model.predict(x_test)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step
```

```
import numpy as np
```

```
predicted_labels = np.argmax(predictions, axis=1)
true_labels = y_test
```

```
from sklearn.metrics import classification_report

report = classification_report(true_labels, predicted_labels)
print(report)
```

```
              precision    recall  f1-score   support

           0       0.99      1.00      0.99       980
           1       1.00      0.99      0.99      1135
           2       1.00      0.99      0.99      1032
           3       0.99      1.00      0.99      1010
           4       0.99      0.99      0.99       982
           5       0.99      0.99      0.99       892
           6       1.00      0.99      0.99       958
           7       0.99      1.00      0.99      1028
           8       0.99      0.99      0.99       974
           9       0.99      0.99      0.99      1009

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```

Start coding or generate with AI.